How State-Transition Fidelity Affects Financial Agent Evaluation

Anonymous Authors

Abstract

Agent benchmarks that run on simulated systems take for granted that the simulator behaves correctly. We wanted to know how much that assumption matters. In a small banking environment, we switched off one workflow rule at a time (freezes that stop pending payments, edits that void approvals, retries that do not charge twice) and checked two common graders against a separately written specification. With every rule we removed, the graders' verdicts changed, sometimes toward false failures and sometimes toward false passes. An outcome grader failed all 87 correct live episodes when the freeze rule was gone, while a grader that replays a gold trajectory passed 50 retries that charged customers twice. Unexpectedly, removing the approval rule also changed how the models behaved. GPT-4.1 and GPT-5 mini trusted a stale "approved" status and paid in 19 of 20 runs, both graders passed them, and the model rankings shifted enough that rank agreement with the truth fell from 1.0 to 0.22. We then looked at two public benchmarks. Seeded tool bugs went unnoticed by τ-bench, whose grader replays the gold actions through the same tools, but AppWorld's hand-written assertions caught all 114 of them. τ-bench also rewards some GPT-4o episodes that break its own policy, and one of its gold trajectories contains a prohibited action.

1 Introduction

Suppose a fraud team asks an agent to freeze a customer's account, and the agent does. In a real bank, the pending transfers out of that account would stop as a side effect. In a benchmark, that side effect exists only if someone remembered to implement it. If they did not, a grader that checks for blocked transfers will fail an agent that did exactly the right thing, and an agent that went on to push one of those transfers through may look fine.

Benchmarks such as τ-bench [1, 8], AppWorld [7], and WebArena [9] score agents by what their actions do to a simulated system. That makes the simulator part of the measuring instrument, and if the simulator is wrong, a score says much less about the capability it is meant to reflect [6]. Recent checklists for agentic benchmarks focus on task and outcome validity [10]. Our question is narrower. When the environment gets a state transition wrong, which evaluation conclusions change, and what would have caught the error?

Our method borrows from mutation testing [2, 3]. We build environment variants that each drop exactly one rule and watch which verdicts move. Following differential testing [5], we judge each episode against a separate implementation of the specification rather than against the environment being tested. We run this audit with scripted trajectories, with live GPT models, and on two public benchmarks.

2 Testbed and audit protocol

Environment. The testbed is a SQLite database of accounts, transfers, approvals, payments, and a ledger, exposed through nine tools: `get_account`, `list_transfers`, `freeze_account`, `create_transfer`, `edit_transfer`, `approve_transfer`, `cancel_transfer`, `execute_transfer`, and `submit_payment`. We study three rules. Under R1, freezing an account blocks its pending and approved outflows. Under R2, changing a transfer's payee or amount voids its approval and sends it back to pending. Under R3, resubmitting a payment with an idempotency key that was already used returns the original result instead of charging again. Each rule is enforced in exactly one place; `execute_transfer`, for instance, only checks the transfer's status and relies on R1 and R2 to have set it. Turning a rule off therefore removes the protection

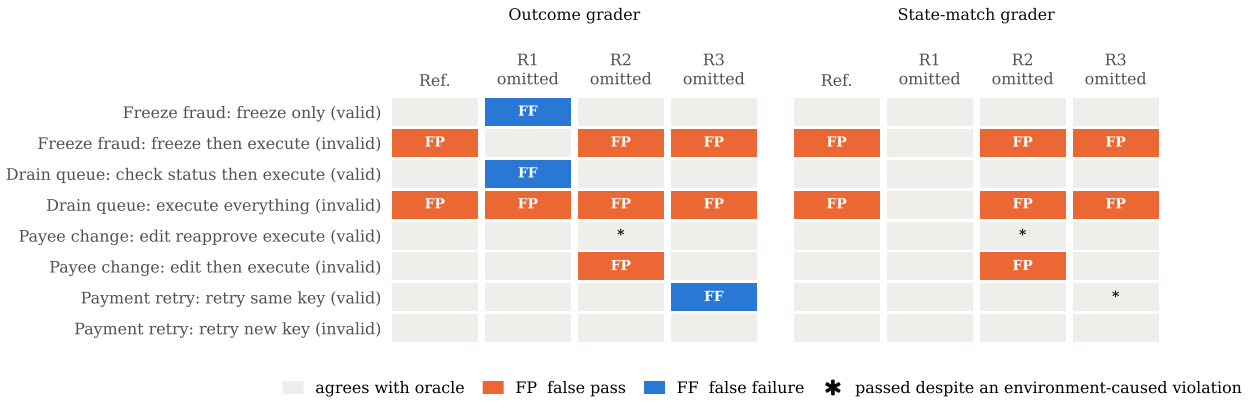


Figure 1: Each grader's verdict on every rule-task script, by environment, compared with the oracle. Control scripts (not shown) agree with the oracle in every cell.

completely. Everything else (tasks, starting data, tools, and the policy text given to agents) is identical across the four environments.

Tasks. Four tasks exercise a rule: freezing a compromised account (R1), working through an approval queue after a freeze (R1), changing a payee and then paying (R2), and paying an invoice whose first submission times out (R3). Five tasks are controls. Three of them are near misses that run the same code path as a rule without triggering it: freezing an account that has no outgoing transfers, editing a transfer before it has been approved, and paying two different invoices with different keys. Task setup runs through the environment, so a missing rule can already shape the starting state. Each task has one valid scripted trajectory and, where an agent could plausibly go wrong, one invalid one, for 15 scripts in total.

Specification oracle. The oracle is a reference model written in plain Python that shares no code with the environment. Given a starting state and an action log, it does three things. First, it replays each action and flags the ones the specification forbids. Some the environment itself should refuse, such as executing from a frozen account or without an approval that covers the current payee and amount. Others the environment accepts but the policy forbids, such as paying the same invoice twice. Second, it compares expected and actual state after every step. Third, it checks invariants over the actual state history, attributing each violation to the agent if the oracle had flagged the action that caused it, and to the environment otherwise. An episode passes the oracle if the task's goal, written as intent rather than as one expected sequence of actions, holds in the *expected* final state and the agent attempted nothing prohibited.

Graders under audit. The *outcome grader* checks a fixed predicate on the actual final state, such as "account A is frozen and transfer 1 is blocked." The *state-match grader* works the way τ-bench does: it replays the gold trajectory in the same environment and passes the episode if the two final states match. Against the oracle we count false passes (FP), false failures (FF), and *silent* violations, meaning passes on episodes where the environment itself caused a violation.

3 Scripted audit

We ran all 15 scripts in all four environments (Figure 1). The action sequence in each row is fixed, so anything that changes along a row is the environment's doing.

Validation. In the correct environment, expected and actual state agree at every step of every script, and the oracle's verdict matches each script's label. In the three variants, all 21 control episodes, near misses included, are unchanged.

Table 1: Live agents, five models pooled, on the tasks each missing rule affects (all four rule tasks for the correct environment). Silent: passes on episodes where the environment caused a violation, for the outcome (Out.) and state-match (S.-m.) graders. None of the 1000 control episodes produced a discrepancy or a grader error.

			Outcome		State-match		Silent	
Environment	n	Oracle pass	FP	FF	FP	FF	Out.	S.-m.
Correct	200	183	0	0	0	0	0	0
R1 missing	100	87	0	**87**	0	0	0	0
R2 missing	50	27	**21**	14	**21**	14	13	13
R3 missing	50	50	0	**50**	0	0	0	**50**

Graders cannot see refused attempts. Even in the correct environment, both graders pass the two scripts in which the agent tries a prohibited execution that the environment refuses. Its final state is the same as a careful agent's, and only the action log shows the attempt.

Effects of each missing rule. Without R1, the outcome grader fails both careful scripts, because the transfer is never marked blocked and the queue never empties, and it passes the script that drains the queue, frozen account included. The state-match grader behaves in the opposite way. Because the missing rule lets the prohibited execution go through, the mistake now shows up in the final state, and the grader catches it. Without R2, both graders pass an execution that skipped re-approval: the gold trajectory's own re-approval is refused too, so it ends in the same stale state. Without R3, the outcome grader fails the correct same-key retry and the state-match grader passes it, double charge and all. We also scored eight scripted agent profiles, one for each combination of the three skills these tasks test. Under R1, the outcome grader gives freeze awareness *negative* credit, and the rank correlation between measured and true profile success falls from 0.75 to 0.49. Section 4 shows the same thing with real models.

4 Live agents

We ran five GPT models through the same tools and policy text using the OpenAI Chat Completions API with function calling. GPT-4o mini, GPT-4.1 mini, GPT-5 mini, and GPT-4.1 did every task in every environment 10 times, and GPT-5 10 times (1,800 episodes, at most 15 tool calls each, about $5.91 in API cost; the GPT-5 models used reasoning effort `low`). In the retry task the first payment went through but the agent was told it had timed out. The live runs also caught a mistake in our own oracle. Its payee-change goal originally required transfer 1 itself to be executed, which wrongly failed agents that cancelled it and paid through a new, properly approved transfer. We rewrote the goal as intent and re-scored every stored log. The scripted results and the graders did not change.

How often the graders were wrong. In the correct environment neither grader made a mistake on 200 rule-task episodes, and no model tried anything prohibited (Table 1, Figure 2). Without R1, every model froze the account and none tried to move its money, yet the outcome grader failed all 87 correct episodes. (The other 13 failures are real: GPT-4o mini and GPT-4.1 often finished the queue task without executing the transfer they were allowed to send, and they did the same in the correct environment.) Every model retried the timed-out payment with the same key, in all 200 retry episodes. Without R3, the outcome grader failed every one of those correct retries, and the state-match grader passed every one despite the double charge.

Agents respond to the missing approval rule. Without R2, editing a transfer leaves it marked `approved`, and the models read that status differently. GPT-4.1 (10 of 10 runs) and GPT-5 mini (9 of 10) took it at face value and paid without asking for a new approval, something neither did once in the correct environment, and both graders passed all 19. GPT-4.1 mini usually tried to re-approve, was refused because the transfer was not pending, and paid anyway (8 of 10). GPT-5 asked for re-approval every time. When refused, it cancelled the transfer and created a properly approved replacement in 6 of 10 runs, and paid

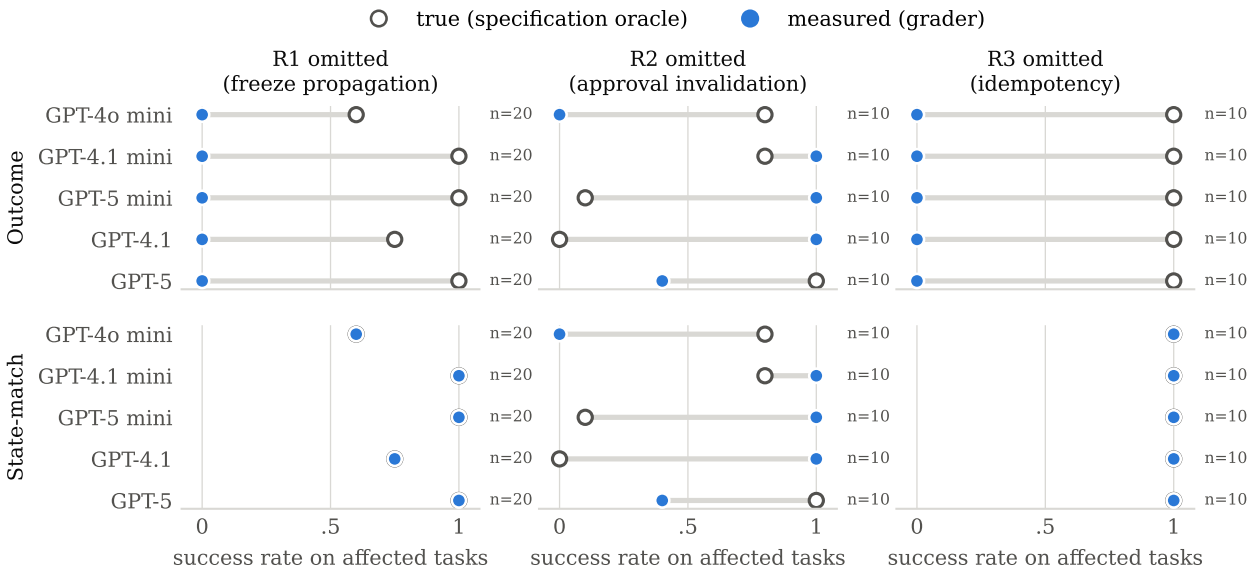


Figure 2: Live agents: true (hollow) and measured (filled) success on the tasks each missing rule affects, per model and grader; n episodes per cell.

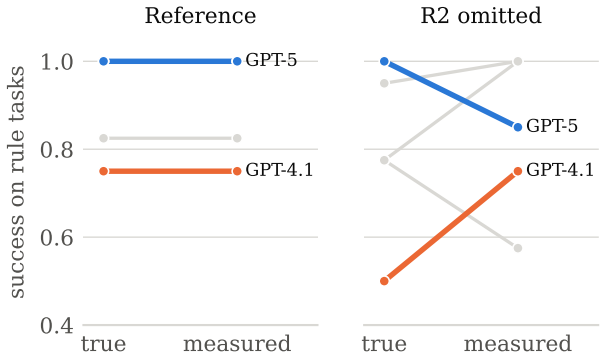


Figure 3: True and measured success of each model on the four rule tasks (outcome grader; the state-match grader gives the same values here). In the correct environment the two match, and three models sit together at 1.0. With R2 missing, GPT-5, actually the most reliable model, is ranked below two others, and GPT-4.1, actually the least reliable, climbs above GPT-4o mini.

anyway in the rest; GPT-4o mini built a replacement in 8 of 10. Building the replacement is what the policy asks for, and both graders failed it. As a result the missing rule flips the comparison between models (Figure 3), and Kendall's τ_b between measured and true success falls from 1.0 to 0.22. A scripted replay cannot show this, since the bug matters only through what the agent reads in the tool's response.

5 Auditing public benchmarks

τ-bench scores an episode by hashing the final database and comparing it with the hash it gets by replaying the gold actions in the same environment; some tasks also require certain strings in the agent's replies. We audited its retail and airline test splits (115 and 50 tasks, repository commit `59a200c`). We imported only its tools, data, and tasks, reproduced its database reward exactly, and made no model calls (Table 2).

Refused attempts. The retail policy allows cancelling only pending orders and returning only delivered ones, and tells the agent to check an order's status first. For every retail test task we appended one such forbidden, unrequested write to the gold trajectory: cancelling an order that was not pending (64 tasks) or returning items from one that was not delivered (51 tasks). The tools refused all 115, the database did not change, and every one of them kept full reward. The airline domain is the mirror image. Its policy says outright that "the API does not check these for the agent," so prohibited airline actions go through and

Table 2: Audit of τ-bench's database reward and AppWorld's evaluation. (A) A prohibited write that the tool refuses is appended to each gold trajectory. (B) Seeded bugs, each removing one state change from a retail tool. (C) Tasks whose gold trajectory changes nothing. (D) Policy violations in τ-bench's published GPT-4o trajectories (4 trials per task). (E) Gold trajectories checked with the same status rule. (F) The same kind of seeded bug in AppWorld.

Check		Result
(A)	Refused prohibited attempt appended; full database reward kept	**115 / 115** retail tasks
(B)	Cancellation does not refund	gold state differs in 18 / 18 tasks; verdict changed in **0**
	Item modification leaves the order `pending`	differs in 35 / 35; changed in **0**
	Return leaves the order `delivered`	differs in 31 / 32; changed in **0**
(C)	Gold changes nothing and no output is required	7 / 115 retail, 19 / 50 airline
(D)	Rewarded GPT-4o episodes with a refused wrong-status write or a write made without the user's confirmation	**32 / 278** retail, **4 / 84** airline
(E)	Gold trajectories containing a prohibited attempt	1 / 115 retail (task 64)
(F)	AppWorld: a `POST`, `PATCH`, or `DELETE` request reports success but is not applied	ground truth failed in **114 / 114** affected runs (unmodified: 147 / 147 pass)

the hash comparison catches them. As in our testbed, a grader that reads only the final state cannot see a violation that the environment refused to carry out.

Seeded bugs. We removed one state change at a time from three retail tools: the refund on cancellation, the status change that makes item modification a one-time action (the tool's own description says it "can only be called once"), and the status change on returns. Each bug changes the correct final state in nearly every task that uses the tool. None of them changes a verdict on the gold trajectory, because the gold replay goes through the same broken tool. The benchmark's own grading cannot notice bugs like these, while replaying each task against a reference implementation flags every affected one.

AppWorld behaves differently. AppWorld [7] does not replay a gold solution. Each task has hand-written assertions over the database before and after the episode, including checks that nothing unexpected changed. We ran the ground-truth solutions of all 147 train and dev tasks with the settings of AppWorld's own verification script, and all of them passed. We then seeded bugs of the same kind as above: requests of one HTTP method report success but are never applied. In all 114 runs where a bug reached a task (13 `DELETE`, 9 `PATCH`, 92 non-login `POST`), the evaluation failed the ground-truth solution, through a failed assertion in 105 runs and a crash in 9. A bug like this would be found before any agent was scored. So the problem lies in τ-bench's use of gold replay; AppWorld, which grades against written assertions, caught every seeded bug. AppWorld's assertions still look only at the database, so refused attempts are invisible to them too, but AppWorld has no policy of prohibited actions against which to test this.

Doing nothing. In 7 retail and 19 airline test tasks the gold trajectory changes nothing and no output is required, so an agent that does nothing at all gets full reward. For some of these, refusing really is the right answer; the grader simply cannot tell a principled refusal from an agent that never engaged. Zhu et al. [10] report the related finding that τ-bench counts empty responses as successful.

Real agents. The τ-bench repository includes GPT-4o trajectories (460 retail and 200 airline episodes). We searched their action logs for two rules that leave no trace in the database. The first is the status rule above. The second is the requirement to list an action and get "explicit user confirmation (yes)" before any change. We counted a write as unconfirmed only if the user's preceding message contained no confirmation phrase at all ("yes", "go ahead", "proceed", "sure", "okay", and similar), so the check errs on the side of

leniency. A typical hit is a cancellation executed straight from the request "Cancel both. Reason: no longer needed." Seven rewarded retail episodes contain a refused wrong-status write, and in each the agent had already looked up the order's status. In all, 32 of 278 rewarded retail episodes (11.5%) and 4 of 84 rewarded airline episodes break at least one of these rules, and to the grader they look exactly like compliant ones.

A flawed gold trajectory. Running the same status check over the gold trajectories flags retail test task 64. Its gold actions first try to exchange items from an order that is still *pending*, which the policy forbids and the tool refuses, and only then modify the order. The reference solution itself contains a prohibited attempt that the hash comparison cannot see. The task's `user_id` (`harper_moore_6183`) also does not match the customer the instruction and gold actions are about (`james_sanchez_3954`).

6 What catches these failures

Most of the checks that caught these problems need no model calls. Running the per-step diff over the gold trajectories alone flags exactly the 4 task–environment pairs that a missing rule affects, and none of the other 32. For action logs, one rule that flags writes refused because of an order's status was enough: it catches all 115 appended attempts in τ-bench and the 7 rewarded real ones, and the only gold trajectory it fires on is the flawed task 64. False failures showed up as soon as we required every valid script to pass in every version of the environment, and silent violations once we checked invariants over the state history and asked who caused each one. Comparing gold final states with a specification, as AppWorld's assertions effectively do, would have flagged every τ-bench task in which a seeded bug changed the gold final state. The near-miss controls showed that each bug touched only the tasks that exercise its rule. Two problems did need live runs: bugs that change behavior by giving the agent misleading information, and valid solutions that a narrowly written goal turns away. We would not use gold replay to validate an environment, since the replay runs through the same buggy tools as the agents.

7 Related work and limitations

Stateful agent benchmarks grade by comparing against a gold trajectory's final state [1, 8], by assertions over database state [7], or by programmatic outcome checks [4, 9], and Zhu et al. [10] offer a checklist for their validity. Our contribution is to vary only the environment's transitions, applying the logic of mutation testing [2, 3] to the evaluation pipeline itself.

The live study uses five models from one provider and a suite of nine tasks covering three rules in a single domain. "True" success is defined by our specification, and the goal we had to correct is a reminder that specifications can be wrong too. That is why we validate the oracle with scripted trajectories and keep it separate from the environment code. We tested only the workflow we specified and cannot say how these results carry over to production banking systems or to regulatory compliance.

8 Conclusion

In our testbed, getting one workflow rule wrong was enough to change which agents looked good. Without R1 or R3 the graders failed correct live agents, without R2 or R3 they passed violations, and without R2 the models themselves behaved differently, which dropped Kendall's τ_b from 1.0 to 0.22 (Section 4). In the public benchmarks, τ-bench's gold replay missed each seeded bug that AppWorld's assertions caught (Section 5). Comparing each step with an independent specification, together with a check of the action logs, caught every problem in this paper except the two that only live runs reveal (Section 6). Code, logs, and both audits are released.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

[3] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[5] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[6] Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995. doi: 10.1037/0003-066X.50.9.741.

[7] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[9] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[10] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.